Ingeniería del Software II

Unidad 1. Tema 4.

Diseño Orientado a Objetos.

Tema 4. Diseño Orientado a Objetos.

- De los Requisitos al diseño. Objetos Y Clases. Proceso del Diseño Orientado a Objetos. Contexto del Sistema y Modelos de Utilización. Diseño de la Arquitectura. Identificación de Objetos. Modelo de Diseño. Especificación de la Interfaz de Objetos. La documentación del Diseño y la importancia de la trazabilidad. Objetos Concurrentes. *Evolución del diseño. Diseño de Software de Tiempo Real. Diseño de Interfaces de Usuario.*

Análisis y Diseño orientado a objetos.

- UML es un lenguaje de modelado unificado.
- Es un lenguaje que permite modelar, construir y documentar los elementos que conforman un sistema software orientado a objetos.
- Un modelo representa a un sistema de software desde una perspectiva específica, es decir cada modelo nos permite fijarnos en un aspecto distinto del sistema.

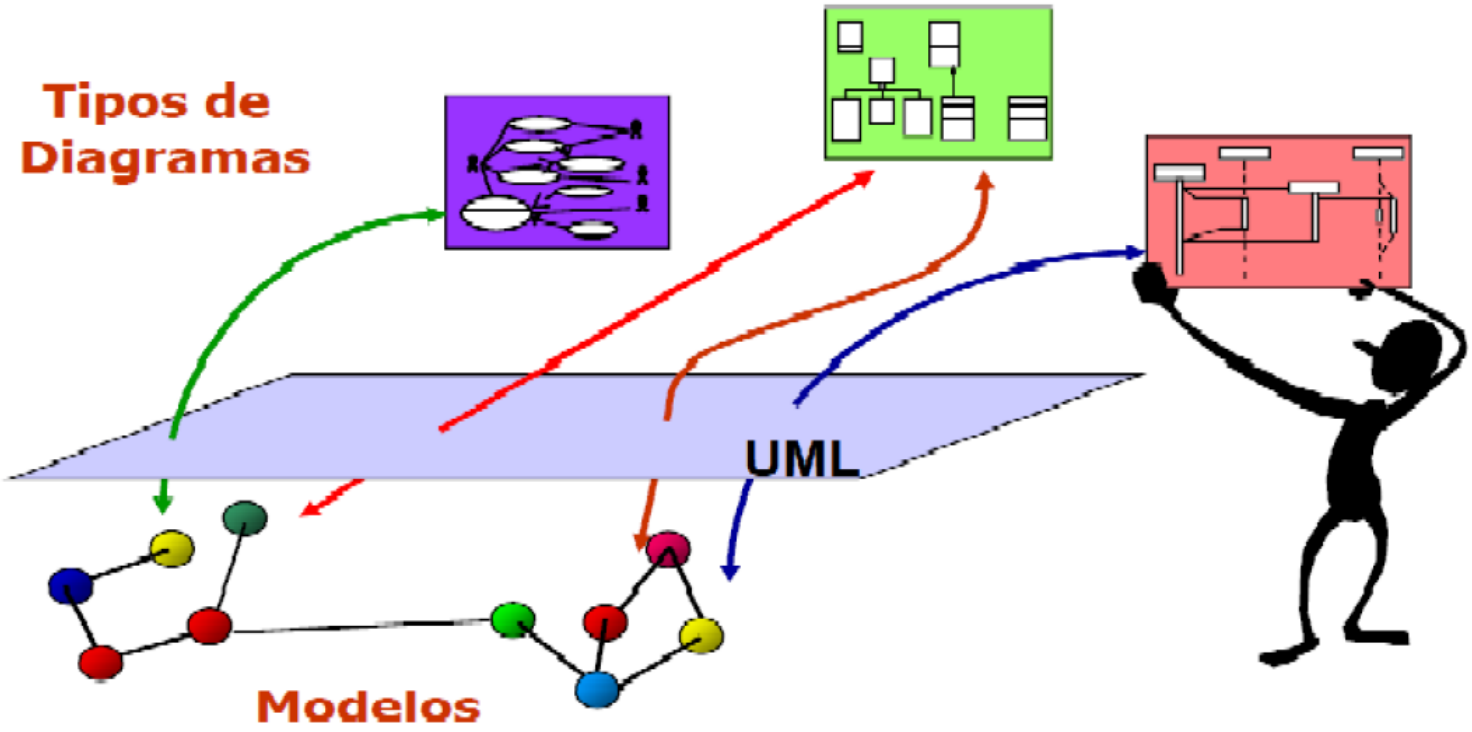
UML

- Es un lenguaje gráfico para visualizar, especificar, construir y documentar las partes de un sistema de software desde distintos puntos de vista.
- Ofrece una forma estándar de modelar sistemas software, pudiendo utilizarse:
  - Con cualquier proceso de desarrollo.
  - A lo largo de todo el ciclo de vida.
  - Con distintas tecnologías de implementación
- Puede usarse también en otras áreas, como la ingeniería de negocio, modelado de procesos, etc.

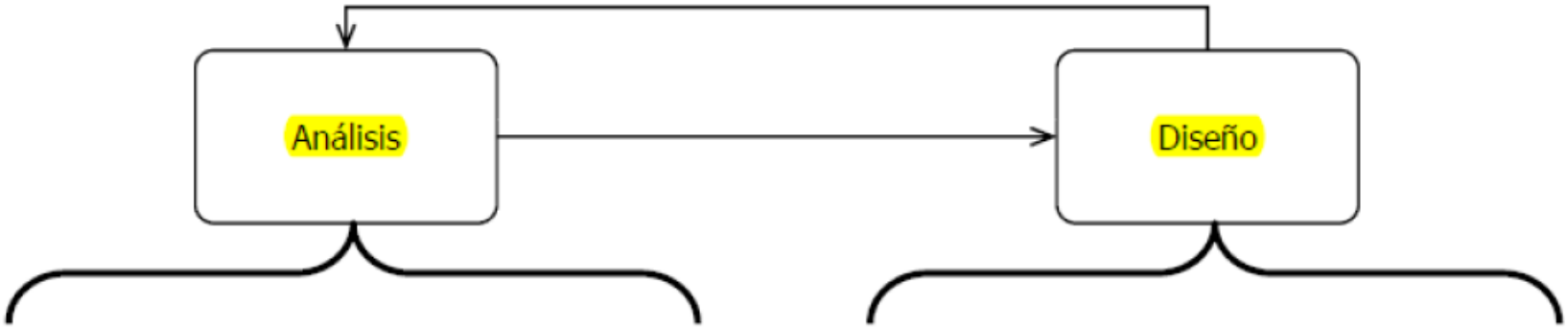
UML

- UML es una familia de notaciones, útiles para describir distintos aspectos de un sistema:
  - *Estático. Describe los elementos del sistema y sus* relaciones.
  - *Dinámico. Describe el comportamiento del sistema a* lo largo del tiempo.

UML



Tipos de Diagramas



- D. Casos de Uso.
- D. Secuencia del Sistema.
- D. Clases Conceptuales.
- D. Clases Análisis.

- D. Clases y Objetos.
- D. Colaboración.
- D. Secuencia.
- D. Estados.

Tipos de Diagramas

Hay dos grupos principales de diagramas:

- **Diagramas de Estructuras:**
  - Muestran una vista estática del modelo.
  - Enfatiza en los elementos que debe existir en el sistema modelado.
- **Diagramas de Comportamiento:**
  - Muestra una vista dinámica del modelo.
  - Enfatiza en lo que debe suceder en el sistema modelado.

Tipos de Diagramas

- **DIAGRAMAS DE ESTRUCTURA**

1. Diagrama de Clases
2. Diagrama de Componentes
3. Diagrama de Objetos
4. Diagrama de Despliegue o Distribución
5. Diagrama de Paquetes

- **DIAGRAMAS DE COMPORTAMIENTO**

6. Diagrama de Actividad
7. Diagrama de Caso de Uso
8. Diagrama de Estados

- **Diagramas de Interacción: son un subtipo de los diagramas de comportamiento.**

9. Diagrama de Secuencia
10. Diagrama de Comunicación o Colaboración

Del Análisis al Diseño

- El modelo de análisis describe el sistema desde el punto de vista de los actores.
- No contiene información de la estructura interna del istema, esto es del ***cómo.***

Diseño del sistema:

- Objetivos de diseño que se deben optimizar (derivados de los requisitos no funcionales).
- Una *arquitectura software: descomposición en subsistemas,* dependencias entre ellos, etc.

Diseño detallado (de objetos).

- Refinamiento del diseño del sistema.
- Diseño de las clases de la solución, interfaces.

Objetos y Clases

Clase:

- Describe grupos de objetos con propiedades (atributos) similares, comportamiento (operaciones) comunes, relaciones con otros objetos y semántica común.
- Cada objeto sabe cuál es su clase, ya que es una instancia de la misma.

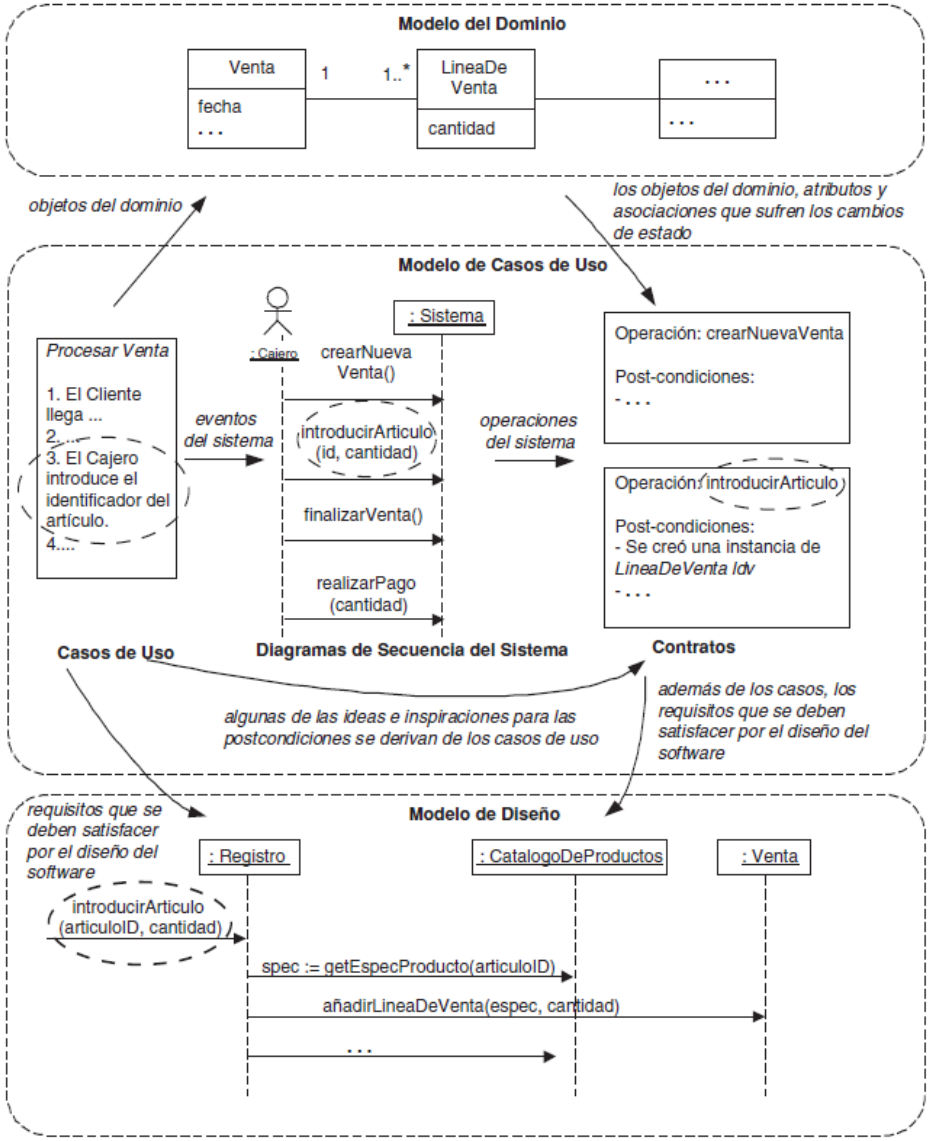
Cuestiones por resolver

- 1.Comprender y definir el contexto y las interacciones externas con el sistema.
- 2. Diseñar la arquitectura del sistema.
- 3. Identificar los objetos principales en el sistema.
- 4. Desarrollar modelos de diseño.
- 5. Especificar interfaces.
  - Recordar la naturaleza creativa del proceso, por lo tanto será iterativo…no secuencial. (I. Somerville)

Primeras fases del proceso de diseño

1. **Modelos de subsistema**, que exponen los agrupamientos lógicos de objetos en subsistemas coherentes. Se representan mediante una forma de diagrama de clase en el que cada subsistema se muestra como un paquete con objetos encerrados. Los modelos de subsistema son modelos estáticos (estructurales).
2. **Modelos de secuencia**, que ilustran la secuencia de interacciones de objetos. Se representan mediante una secuencia UML o un diagrama de colaboración. Los modelos de secuencia son modelos dinámicos.
3. **Modelos de máquina de estado**, que muestran cómo los objetos individuales cambian su estado en respuesta a eventos. Se representan en UML a través de diagramas de estado. Los modelos de máquina de estado son modelos dinámicos.

Del Análisis al Diseño

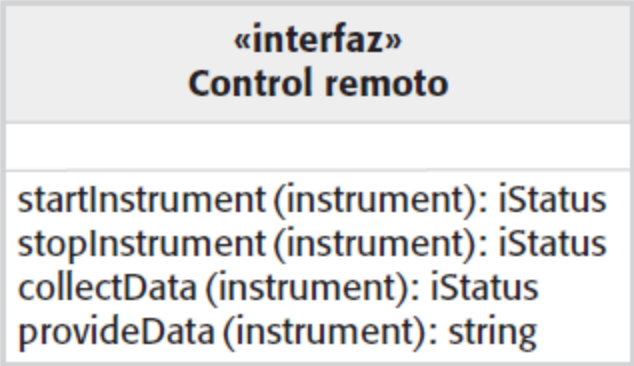


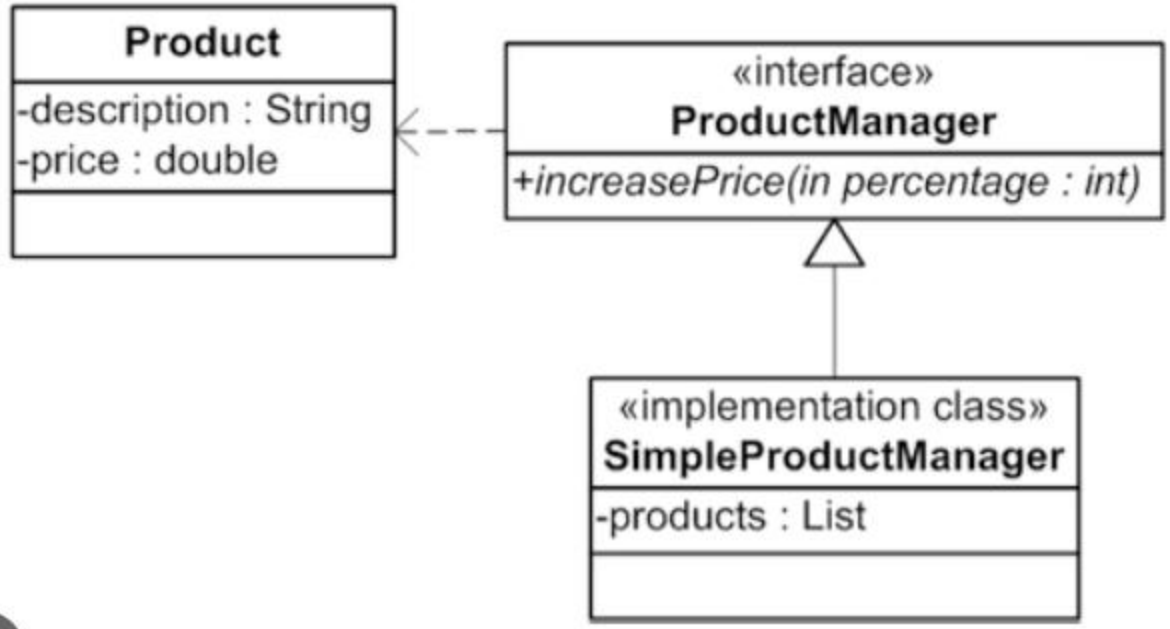
En la práctica, la creación de los diagramas de interacción y clases tiene lugar en paralelo y con sinergia entre ellos. (C.Larman)

Especificación de Interfaz

- El diseño de interfaz se preocupa por la especificación del detalle de la interfaz hacia un objeto o un grupo de objetos. Esto significa definir las firmas y la semántica de los servicios que ofrecerá el objeto o un grupo de objetos. Las interfaces pueden especificarse en el UML con la misma notación de un diagrama de clase. Sin embargo, no hay sección de atributos y debe incluirse el estereotipo UML «interface» en la parte del nombre.
- La semántica de la interfaz se define mediante el lenguaje de restricción de objeto (OCL)

Especificación de Interfaz





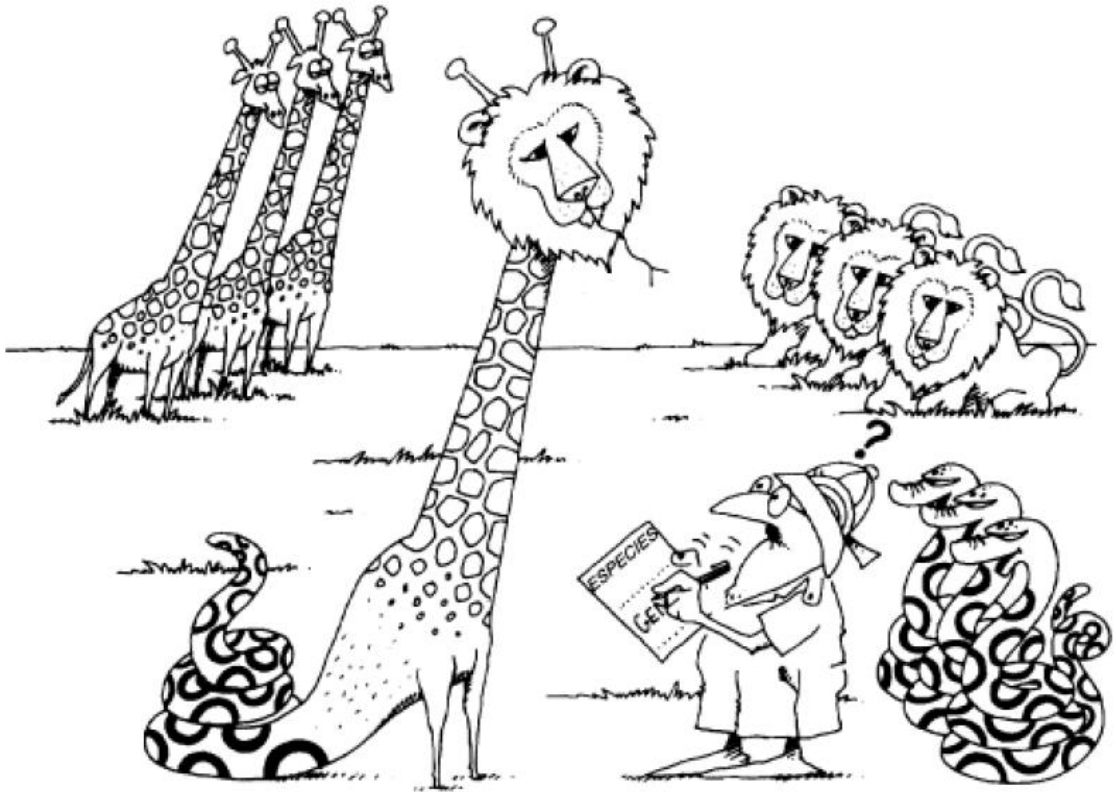
Definición de una clase

- Identificar y nombrar la clase
- Identificar sus componentes
- Identificar sus atributos
- Identificar los errores
- Identificar las conexiones funcionales (qué clases sirve/exige)
- Definir conexiones con superclase y subclases
- Identificar propiedades especiales (persistencia, concurrencia)
- Probar la clase en un prototipo.

Identificar atributos

- El conjunto de atributos de una clase debe ser:
  - Completo (contienen toda la información pertinente)
  - General (se aplican a todos los objetos de la clase)
  - Diferenciado (cada atributo representa un aspecto diferente de la clase)

Suele pasar….



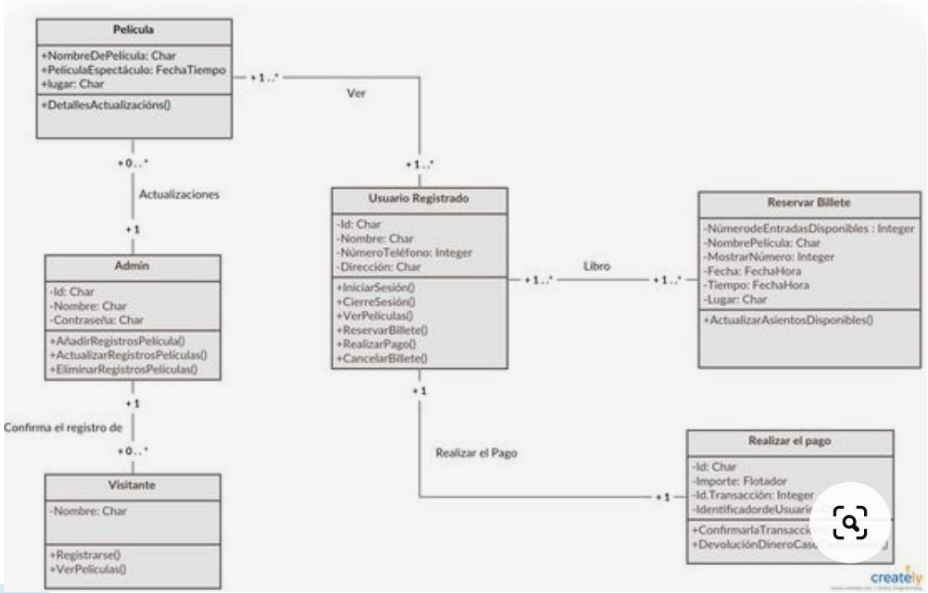
La clasificación es el medio por el cual ordenamos el conocimiento.

Definir atributos

- Tipos de atributos
  - Atómicos predefinidos (entero, real, carácter, pixel...)
  - Atómico enumerativo (color, día de la semana...)
  - Colección
  - Composición (referencias objetos)
- Valor del atributo
  - Común a muchos objetos (variable de clase)
  - Propio de un objeto (variable de objeto)

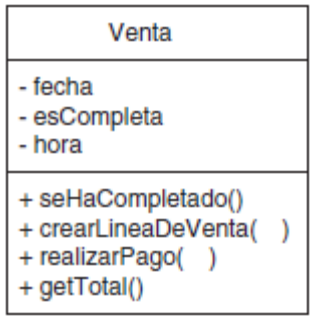
Modelo de Objetos y DCC

- El Modelo de Objetos utiliza un Diagrama de Configuración de Clases (DCC) para definir y mostrar la estructura y comportamiento de todas las clases identificadas en el dominio del problema, así como sus relaciones. El DCC es un modelo semántico extendido.



Crear el diagrama de Clases de Diseño–

- El primer paso en la creación de los DCD como parte del modelo de la solución es identificar aquellas clases que participan en la solución software. Se pueden encontrar examinando todos los diagramas de interacción y listando las clases que se menciona.
- Se pueden identificar los nombres de los métodos analizando los diagramas de interacción.



Operaciones y Métodos

- Función o transformación que se aplica ‘a’ o ‘por’ los objetos en una clase.
- Tienen un objeto destino como argumento implícito (el objeto actual).
- Polimorfismo: la misma operación puede aplicarse a clases diferentes dentro de una jerarquía de herencia.
- Método: implementación de una operación para una clase. Las operaciones con métodos en varias clases deben compartir la misma signatura (tipos de sus argumentos y valor que devuelven).

Identificar Métodos

- Método: algoritmo que utiliza y modifica los atributos de una clase
- Un método es desencadenado por un mensaje
- Funcionalidad de la clase: conjunto de sus métodos.
- El conjunto de métodos debe ser:
  - Completo (realizan toda la funcionalidad de la clase)
  - General (se aplican a todos los objetos de la clase)
  - Diferenciado (cada método debe ser simple y realizar una sola función)

Definir Métodos

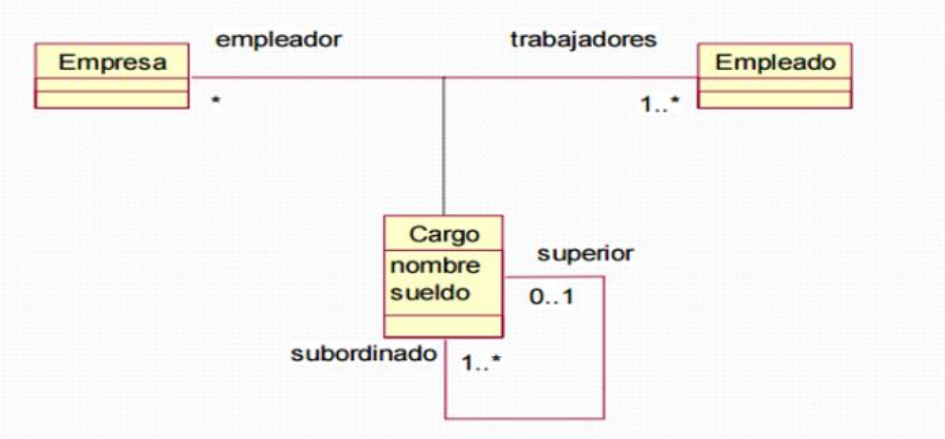
- Tipos de métodos
  - Modificador (asigna valor a un atributo)
  - Selector (devuelve el valor de un atributo)
  - Aplicable a la clase (constructor)
  - Aplicable al objeto
- Parámetros del método
  - ¿Qué información necesita? (argumentos de entrada)
  - ¿Qué debe devolver? (resultado y argumentos salida)

Identificar Errores

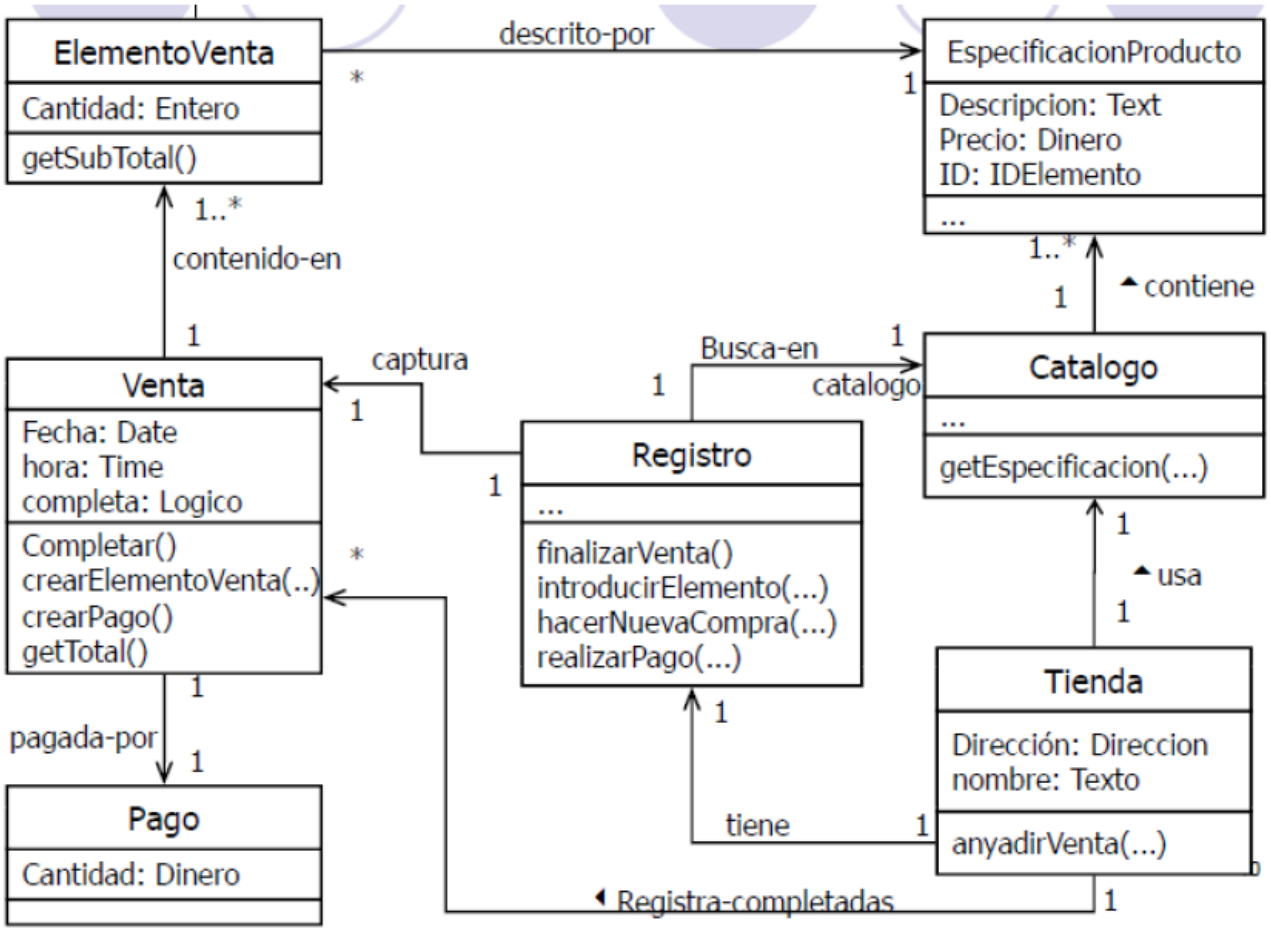
- ¿Qué puede salir mal durante la ejecución de un método?
- ¿Qué comprobaciones debe hacer cada método?
- ¿Cómo interceptar y corregir las condiciones de error?

Enlaces y Asociaciones

- Enlace: conexión física o conceptual entre objetos.
- Asociación: grupo de enlaces con la misma estructura y semántica común.
- Sentido de una asociación:
  - Inherentemente son bidireccionales. Directa (*forward) e inversa (inverse).*

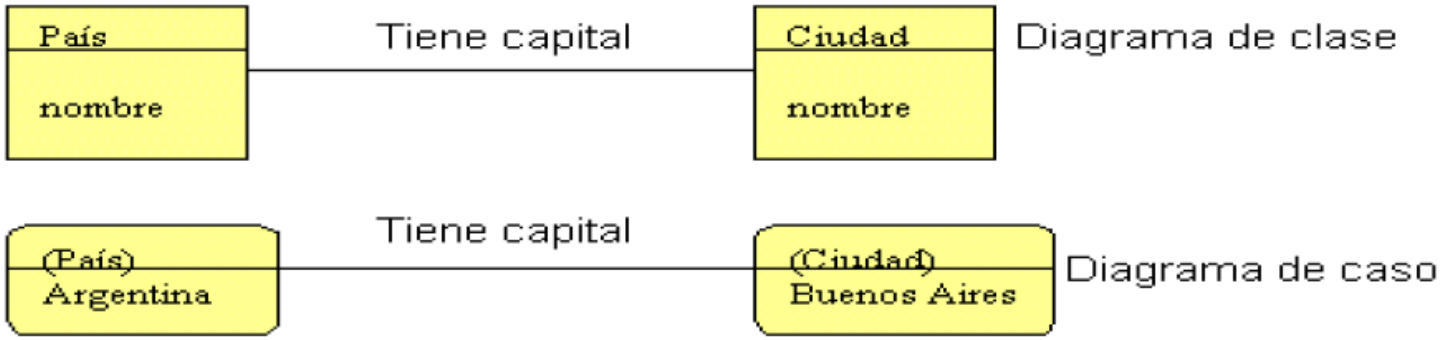


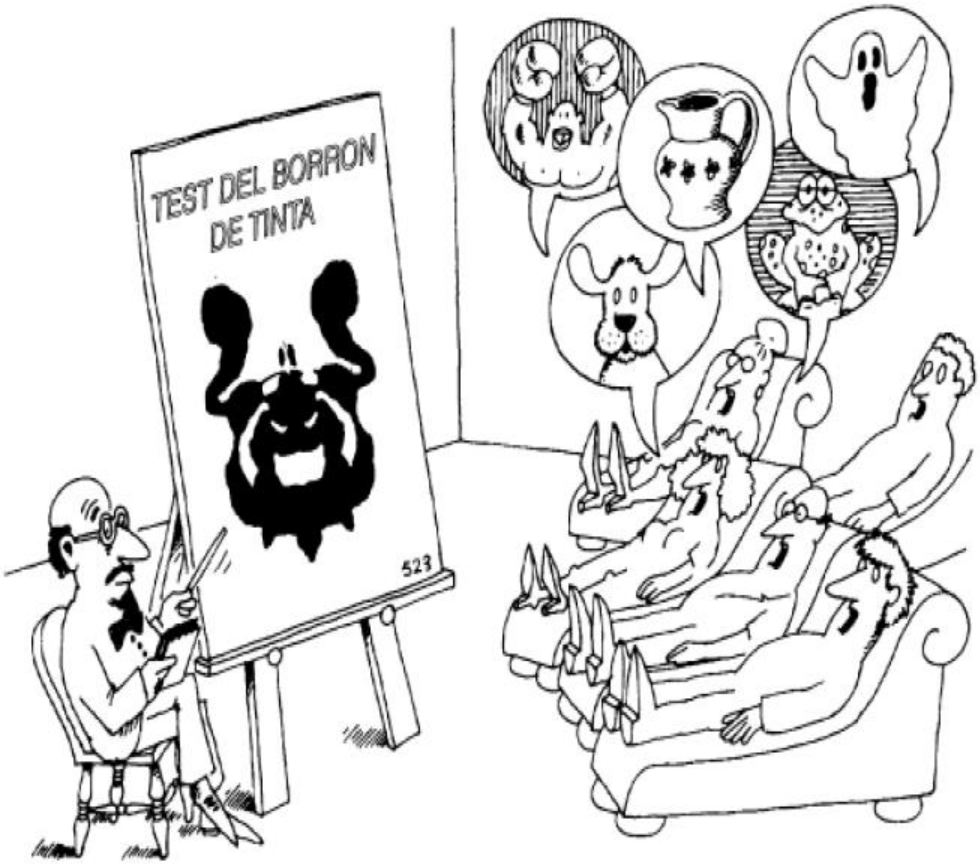
Ejemplo


ElementoVenta
Cantidad: Entero
getSubTotal()
descrito-por
*
1
EspecificacionProducto
Descripcion: Text
Precio: Dinero
ID: IDElemento
...
1..*
contenido-en
1
1..*
1
contiene
Venta
Fecha: Date
hora: Time
completa: Logico
Completar()
crearElementoVenta(..)
crearPago()
getTotal()
captura
1
1
Busca-en
1
catalogo
Catalogo
...
getEspecificacion(...)
Registro
...
finalizarVenta()
introducirElemento(...)
hacerNuevaCompra(...)
realizarPago(...)
1
*
1
usa
1
1
pagada-por
1
Pago
Cantidad: Dinero
1
tiene
1
Tienda
Dirección: Direccion
nombre: Texto
anyadirVenta(...)
Registra-completadas
1

Asociaciones

- Las asociaciones pueden ser binarias, ternarias o de órdenes superiores.
- Los nombres de las asociaciones son opcionales en la notación (se escriben en cursiva).





Diferentes observadores pueden clasificar el mismo objeto de distintas formas.

Multiplicidad

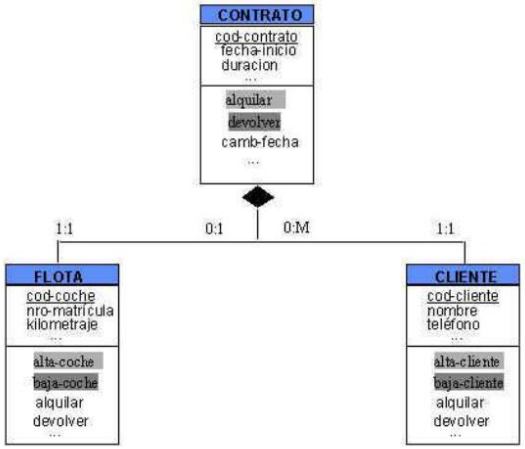
- Especifica cuántos objetos de una clase pueden relacionarse con un único objeto de una clase asociada.
- Se especifica mediante:
  - Un subconjunto (posiblemente infinito) de enteros no negativos: 0..n, o bien uno o varios intervalos no conexos: 1..4, 7.
  - No debemos preocuparnos de ajustar la multiplicidad demasiado pronto en nuestro Modelo de Objetos, si es necesaria su revisión en la etapa de diseño.
  - En los Diagramas de Objetos la multiplicidad se especifica mediante símbolos especiales en los extremos de las líneas de las asociaciones.

Relaciones

- Las acciones son servicios que un objeto puede activar (actuando como agente) para consultar o modificar el estado de otro objeto.
- Las relaciones estructurales que podemos modelar son la agregación (parte-de) y la herencia (es- un).

Agregación

- Se presenta la relación de **Agregación entre clases,** incluyendo información sobre cardinalidades (mínimas y máximas) que determinan cuántos objetos componentes forman parte de un objeto compuesto e inversamente, cuántos objetos compuestos pueden estar compuestos de un objeto en particular.



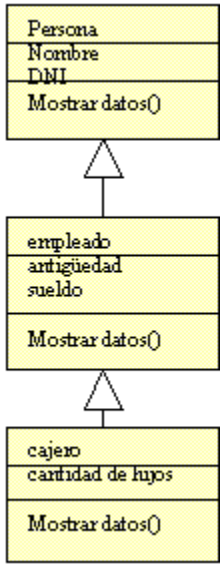
Herencia

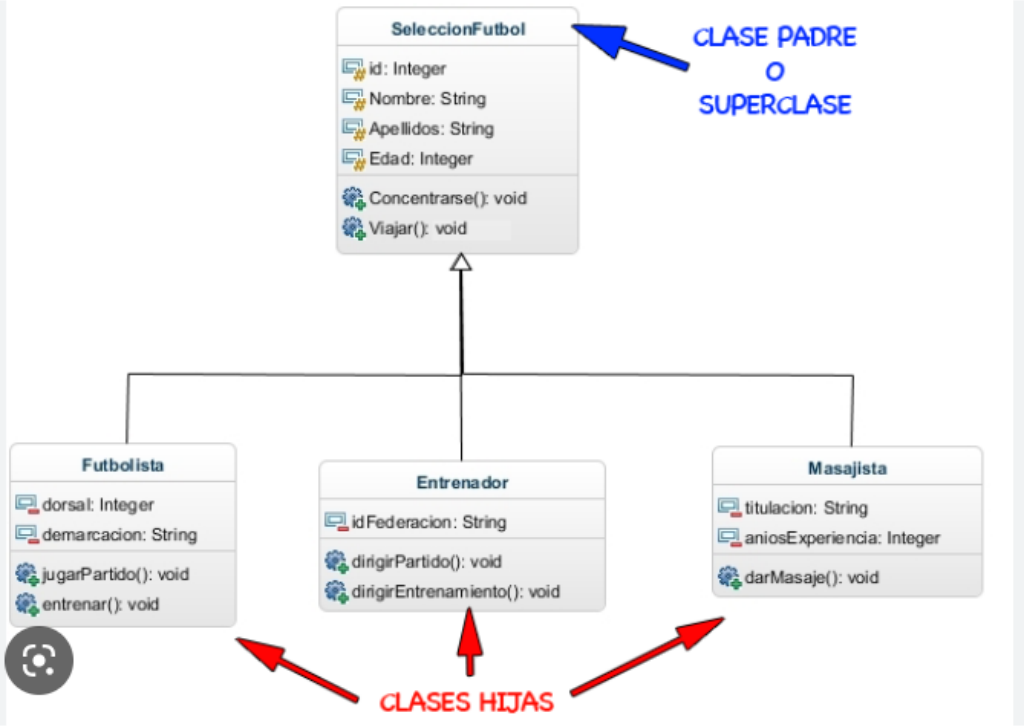
- La **Herencia se define gráficamente trazando una flecha** desde la subclase hacia la superclase correspondiente. Esta flecha de especialización puede etiquetarse con una condición de especialización o con los eventos correspondientes de activación/cancelación de la especialización temporal (rol).

Herencia / Generalización

- Generalización: relación entre una clase (superclase) y una o más versiones refinadas de ella (subclases).
- Relación "es-un".
- Las subclases ‘heredan’ las características, atributos y operaciones de su superclase.
- Una instancia de una subclase es una instancia de sus clases antecesoras o ascendientes.
- Distinción entre generalización y herencia:
  - Generación: relación entre clases.
  - Herencia: mecanismo para compartir características.

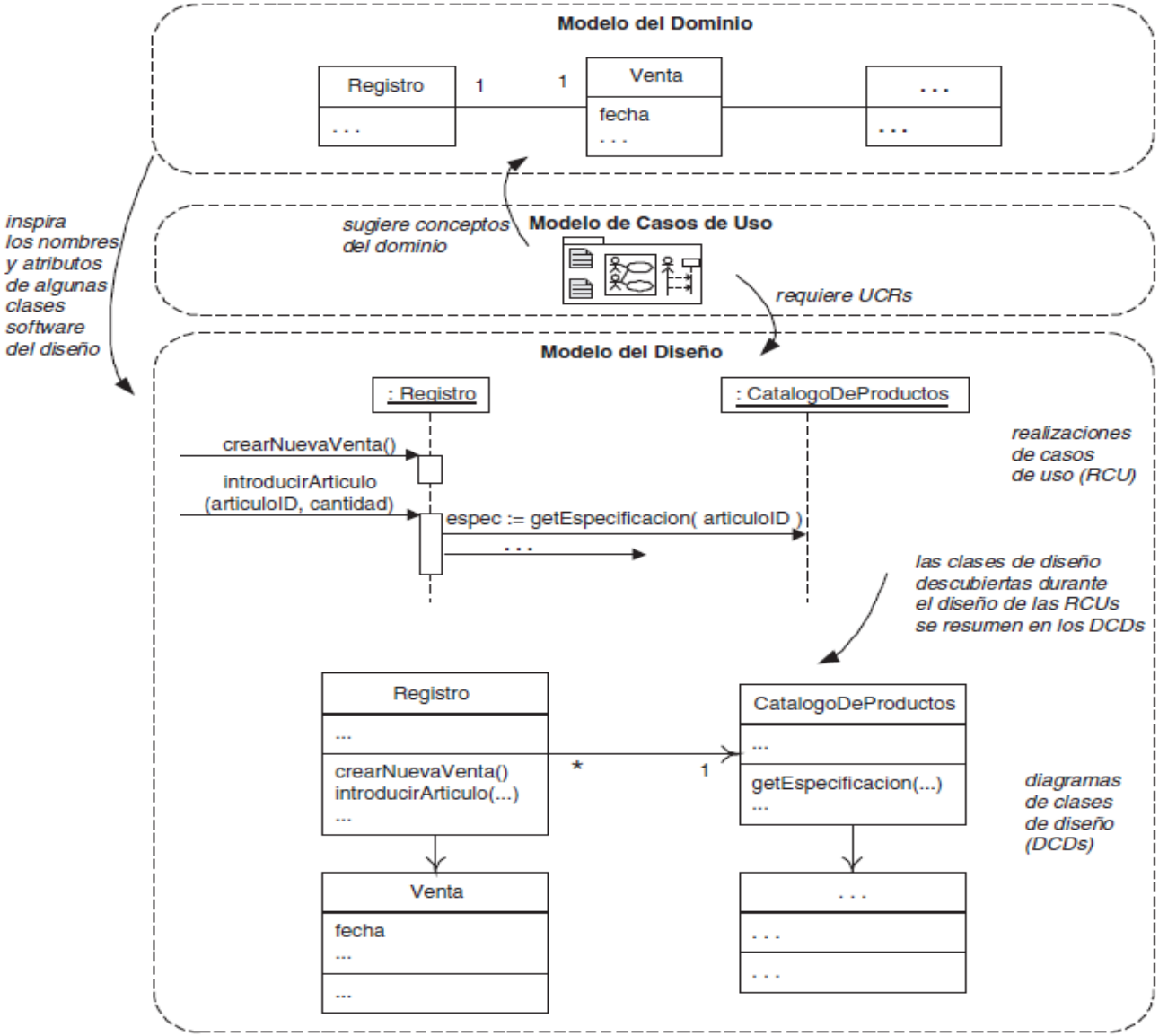
Ejemplos de Herencia





Recordar

- Sobre el diagrama de clases construido revisar:
- Relaciones y cardinalidades entre clases.
- Mantener coherencia con los demás diagramas que forman parte de la documentación del sistema.
- Respetar los criterios de Cohesión y Acoplamiento.
- Al implementar, primero inicio por las clase entidad (menos dependencias) y luego por las controladoras (lógica del negocio)


Modelo del Dominio
Registro
...
1
1
Venta
fecha
...
...
...
inspira los nombres y atributos de algunas clases software del diseño
sugiere conceptos del dominio
Modelo de Casos de Uso
requiere UCRs
Modelo del Diseño
: Registro
: CatalogoDeProductos
crearNuevaVenta()
introducirArticulo (articuloID, cantidad)
espec := getEspecificacion(articuloID)
...
realizaciones de casos de uso (RCU)
las clases de diseño descubiertas durante el diseño de las RCUs se resumen en los DCDs
Registro
...
crearNuevaVenta()
introducirArticulo(...)
...
*
1
CatalogoDeProductos
...
getEspecificacion(...)
...
diagramas de clases de diseño (DCDs)
Venta
fecha
...
...
...
...
...

- Ver desarrollo del ejemplo del Modelo de Venta.

Libro UML y Patrones, Craig Larman; Segunda Edición. Editorial: Prentice Hall.

Capitulo 3 al 15

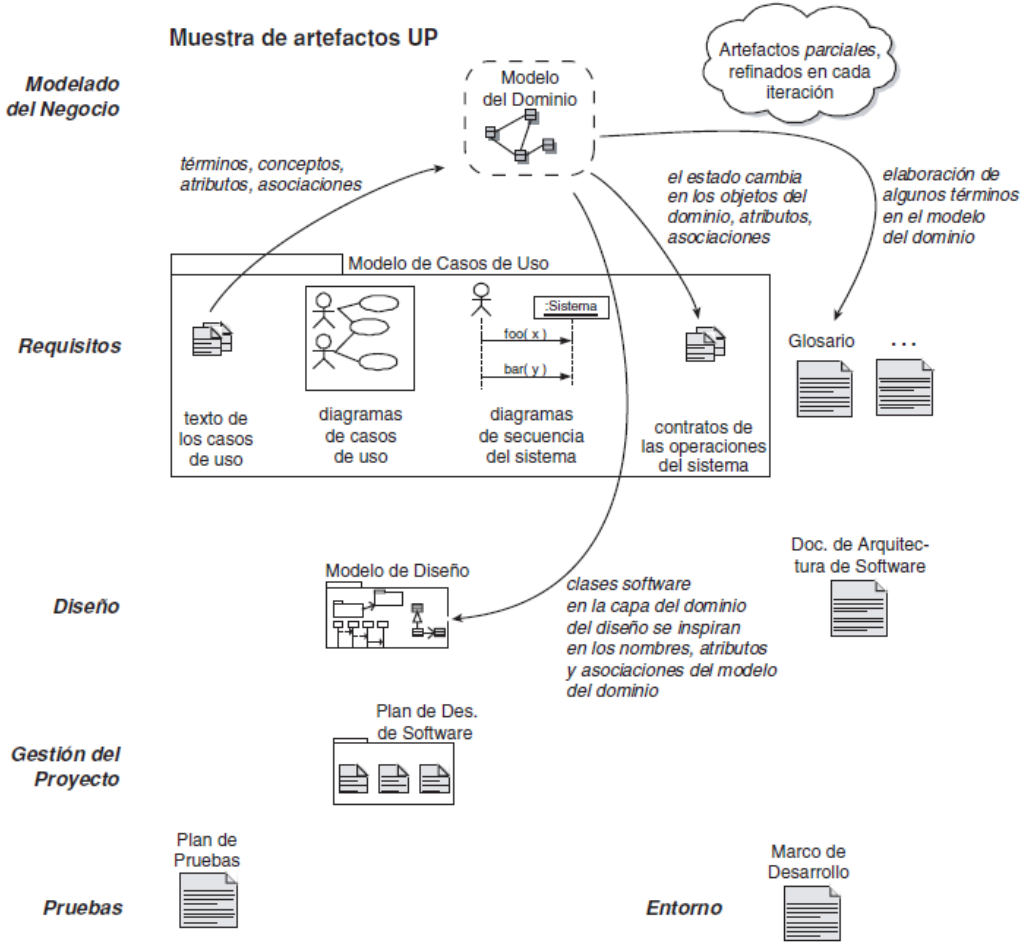


Figura 10.12. Muestra de la influencia entre los artefactos UP.

Parte del DC Conceptual

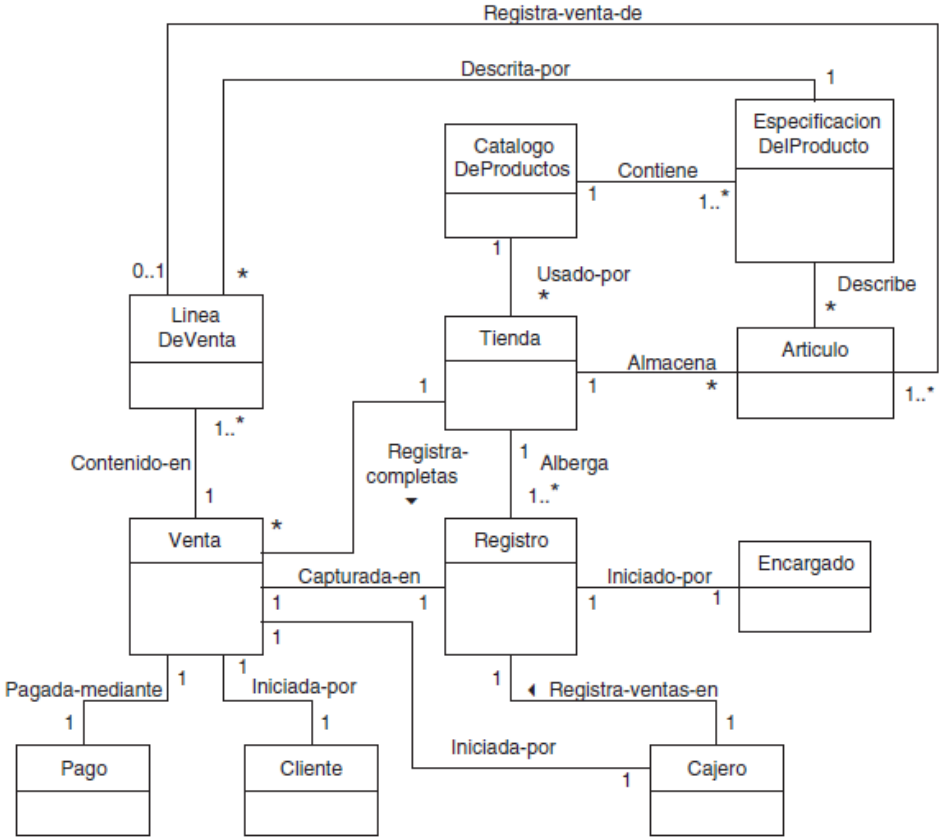


Figura 11.8. Modelo del dominio parcial.

- Actividad para el hogar.
- Ejemplo Aplicación de mensajería integrada.

Bibliografía

- “Modelado y Diseño Orientados a Objetos”, (Metodología OMT), 1996, James
- Rumbaugh, Prentice Hall
- “El Lenguaje Unificado de Modelado”, James Rumbaugh, 2000, Prentice Hall
- UML y Patrones, Craig Larman; Segunda Edición. Editorial: Prentice Hall.
- Ingeniería de software un enfoque práctico’, Roger S. Pressman, 7th ed. Editorial MC Graw Hill.
- ‘Ingeniería del Software’, Ian Sommerville. 9° ed. Editorial Prentice Hall. Material de Cátedra Ingeniería del Software II, Facena UNNE. https://virtual-moodle.unne.edu.ar/login/index.php
- www.exa.unicen.edu.ar/catedras/ui_devlp/papers/DisenioInterfaces.ppt
- J.A. de la Puente DIT/UPM, Diseño de Sistemas de Tiempo Real, material de

presentación.

- http://www.electro.fisica.unlp.edu.ar/temas/p7/RTS-1.html
- V.Saldaño EVOLUCIÓN Y PERSPECTIVAS FUTURAS SOBRE BDOO, Univ. Patagonia Austral
- https://www.youtube.com/watch?v=z807FTdsOgY, consulta al 15/07/2022

Muchas
Gracias
Por
Su atención